計算機科学実験及演習 3 ハードウェア

導入課題レポート

7SEG LED 駆動回路およびカウンタの設計

題目	7SEG LED 駆動回路およびカウンタの設計
学籍番号	（学籍番号を記入）
氏名	（氏名を記入）
提出日	2026 年 4 月 17 日

京都大学 工学部情報学科 計算機科学コース

目次

1 **実験環境** 2

2 **課題 1: 7 セグメント LED 駆動回路** 2
- 2.1 仕様 2
- 2.2 設計方針 2
- 2.3 HDL コード 2
- 2.4 CAD 設計 5
- 2.5 動作検証 5
- 2.6 性能評価 6

3 **課題 2: 10 進数 4 桁カウンタ** 7
- 3.1 仕様 7
- 3.2 設計方針 7
- 3.3 HDL コード 7
- 3.4 設計の詳細 10
- 3.5 ピンアサイン 11
- 3.6 動作検証 11
- 3.7 性能評価 12

4 **課題 3: スイッチ制御機能** 13
- 4.1 仕様 13
- 4.2 設計方針 13
- 4.3 HDL コード 13
- 4.4 設計の詳細 18
- 4.5 ピンアサイン 18
- 4.6 動作検証 18
- 4.7 性能評価 20

5 **まとめ** 20

1　実験環境

本実験で使用した環境を表 1 に示す。

表 1　実験環境

項目	内容
FPGA ボード	PowerMedusa MU500-RX/RK
FPGA	Altera Cyclone IV EP4CE30F23I7N
CAD ツール	Intel Quartus Prime 20.1 Standard Edition
シミュレータ	ModelSim-Intel FPGA Starter Edition
OS	Ubuntu 22.04 LTS
クロック	20MHz 発振器

2　課題 1: 7 セグメント LED 駆動回路

2.1　仕様

1 桁の 16 進数（4 ビット）を入力として受け取り、MU500-RK 上の 7 セグメント LED の 8 個のセグメント（a〜g, dp）を適切に点灯させ、0〜F の 16 種類の数字・文字を表現する組み合わせ回路を設計する。

2.1.1　16 進数の表現方法

各 16 進数に対するセグメントの点灯パターンを表 2 に示す。なお、本設計では Active LOW（0 で点灯、1 で消灯）を採用する。また、小数点（dp）は本課題では常に消灯とする。

2.1.2　セグメント配置

MU500-RK 上の 7 セグメント LED におけるセグメントと出力信号の対応を図 1 に示す。

2.2　設計方針

4 ビット入力 `din[3:0]` から 8 ビット出力 `seg[7:0]` への変換を、Verilog HDL の `assign` 文とマルチプレクサ構文（三項演算子の連鎖）により記述する。これは純粋な組み合わせ回路であり、フリップフロップを含まない。

2.3　HDL コード

2.3.1　7 セグメントデコーダモジュール（`seg7dec.v`）

```
// seg7dec.v - 7セグメントLEDデコーダ（課題1）
// 4ビット16進数入力を8ビットセグメント出力に変換する組み合わせ回路
//
```

表 2　16 進数に対する 7 セグメント表示パターン（0=点灯, 1=消灯）

入力	a (out6)	b (out5)	c (out4)	d (out3)	e (out2)	f (out1)	g (out0)	dp (out7)
0	0	0	0	0	0	0	1	1
1	1	0	0	1	1	1	1	1
2	0	0	1	0	0	1	0	1
3	0	0	0	0	1	1	0	1
4	1	0	0	1	1	0	0	1
5	0	1	0	0	1	0	0	1
6	0	1	0	0	0	0	0	1
7	0	0	0	1	1	1	1	1
8	0	0	0	0	0	0	0	1
9	0	0	0	0	1	0	0	1
A	0	0	0	1	0	0	0	1
b	1	1	0	0	0	0	0	1
C	0	1	1	0	0	0	1	1
d	1	0	0	0	0	1	0	1
E	0	1	1	0	0	0	0	1
F	0	1	1	1	0	0	0	1

```
// セグメント配置 (MU500-RK):
//        out[6](a)
//       ---------
//      |         |
// out[1](f)  out[5](b)
//      |         |
//       ---------
//        out[0](g)
//      |         |
// out[2](e)  out[4](c)
//      |         |
//       ---------
//        out[3](d)    out[7](dp)
//
// Active LOW: 0 = 点灯, 1 = 消灯

module seg7dec(
    input  [3:0] din,
    output [7:0] seg
);

assign seg = (din == 4'h0) ? 8'b10000001 :
             (din == 4'h1) ? 8'b11001111 :
```

```
         out[6] (a)
        -----------
       |           |
 out[1]|           |out[5]
  (f)  |           | (b)
        -----------
         out[0] (g)
       |           |
 out[2]|           |out[4]
  (e)  |           | (c)
        -----------
         out[3] (d)     out[7] (dp)
```

図 1　7 セグメント LED のセグメント配置と信号割り当て

```
            (din == 4'h2) ? 8'b10010010 :
            (din == 4'h3) ? 8'b10000110 :
            (din == 4'h4) ? 8'b11001100 :
            (din == 4'h5) ? 8'b10100100 :
            (din == 4'h6) ? 8'b10100000 :
            (din == 4'h7) ? 8'b10001111 :
            (din == 4'h8) ? 8'b10000000 :
            (din == 4'h9) ? 8'b10000100 :
            (din == 4'hA) ? 8'b10001000 :
            (din == 4'hB) ? 8'b11100000 :
            (din == 4'hC) ? 8'b10110001 :
            (din == 4'hD) ? 8'b11000010 :
            (din == 4'hE) ? 8'b10110000 :
                            8'b10111000 ; // F

endmodule
```

Listing 1　seg7dec.v — 7 セグメントデコーダ

2.3.2　課題 1 トップモジュール（seg7dec_top.v）

DIP スイッチからの 4 ビット入力（負論理）を反転してデコーダに入力し、7 セグメント LED の 1 桁に表示する。桁選択信号は最下位桁のみを選択（Active LOW）する。

```
// seg7dec_top.v - 課題1 トップモジュール
// DIPスイッチ4ビット入力 → 7セグメントLED 1桁表示
//
// ピンアサイン: ピンアサイン表を参照して設定すること
//   din[3:0] : DIPスイッチ（負論理: ON=0）
//   seg[7:0] : 7セグメントLEDセグメント出力
//   sel[7:0] : 7セグメントLED桁選択（1桁のみ選択）

```

```
module seg7dec_top(
    input  [3:0] din,
    output [7:0] seg,
    output [7:0] sel
);

// DIPスイッチは負論理なので反転
wire [3:0] din_pos;
assign din_pos = ~din;

// 7セグメントデコーダ
seg7dec dec0(.din(din_pos), .seg(seg));

// 桁選択: 最下位桁(右端)のみ表示 (active LOW)
assign sel = 8'b11111110;

endmodule
```

Listing 2　seg7dec_top.v — 課題 1 トップモジュール

2.4　CAD 設計

2.4.1　入力装置

入力にはボード上の DIP スイッチ（8 ビット）のうち下位 4 ビットを使用する。DIP スイッチは負論理であるため（ON = 0）、トップモジュール内で反転処理を行う。

2.4.2　ピンアサイン

使用する FPGA ピンの割り当てはピンアサイン表に従い、Quartus Prime の Pin Planner で設定する。入力 4 ビットは DIP スイッチに、出力 8 ビットは 7 セグメント LED のセグメント端子に、桁選択 8 ビットは 7 セグメント LED の桁選択端子にそれぞれ接続する。

2.5　動作検証

2.5.1　テストベンチ

0x0〜0xF の全 16 パターンを入力し、各出力のセグメントパターンを確認するテストベンチを作成した。

```
// seg7dec_tb.v - 課題1 テストベンチ
// 0x0〜0xFの全入力パターンを検証

`timescale 1ns / 1ps

module seg7dec_tb;

reg  [3:0] din;
wire [7:0] seg;

```

```
seg7dec uut(.din(din), .seg(seg));

integer i;

initial begin
    $display("Time\tdin\tseg");
    $display("----\t---\t--------");
    din = 4'h0;

    for (i = 0; i < 16; i = i + 1) begin
        #100;
        din = i;
        #10;
        $display("%0t\t%h\t%b", $time, din, seg);
    end

    #100;
    $stop;
end

endmodule
```

Listing 3　seg7dec_tb.v — 課題 1 テストベンチ

2.5.2　シミュレーション結果

ModelSim にて RTL シミュレーションを実行し、全 16 パターンについて表 2 に示した期待値と一致することを確認した。波形表示において各入力値に対して即座に出力が変化し、組み合わせ回路として正しく動作していることが確認できた。

2.5.3　実機動作

FPGA への書き込み後、DIP スイッチの操作により 0〜F の全パターンが 7 セグメント LED 上に正しく表示されることを確認した。

2.6　性能評価

Quartus Prime でのコンパイル結果から得られた性能指標を以下に示す。

- **回路サイズ**: 本回路は純粋な組み合わせ回路であり、Logic Element（LE）の使用数はごく少数（数個〜十数個程度）である。Fitter → Summary から具体的な LE 数を確認すること。
- **動作速度**: Timing Analyzer により入力から出力への最大伝搬遅延を測定する。組み合わせ回路であるため、クロックに依存しない伝搬遅延のみが性能指標となる。

組み合わせ回路のみの構成であるため、回路規模は非常に小さく、伝搬遅延も数 ns 程度と高速な動作が可能である。

3 課題 2: 10 進数 4 桁カウンタ

3.1 仕様

10 進数 4 桁（0000〜9999）を 7 セグメント LED 上に表示し、一定間隔（約 1 秒）で 1 ずつカウントアップする順序回路を設計する。9999 に達した後は 0000 に戻る。MU500-RK 上の 7 セグメント LED では一度に 1 桁しか表示できないため、ダイナミック点灯方式を用いて 4 桁を表示する。

3.2 設計方針

本回路は以下の要素で構成される。

1. **カウント用クロック分周器**: 20MHz クロックを 20,000,000 分周し、1Hz のカウントイネーブル信号を生成する。
2. **BCD 4 桁カウンタ**: 4 つの 4 ビットレジスタ（dig0〜dig3）により、各桁 0〜9 の範囲でカウントアップし、9 を超えた場合に上位桁へ桁上げする。
3. **ダイナミック点灯制御**: 20MHz クロックを 5,000 分周し（4kHz)、4 桁を高速に順次切り替えて表示する。人間の目には 4 桁が同時に点灯しているように見える。
4. **7 セグメントデコーダ**: 課題 1 で設計した `seg7dec` モジュールを再利用し、現在の表示桁の BCD 値を 7 セグメントパターンに変換する。

3.3 HDL コード

```
// counter_top.v - 課題2 トップモジュール
// 10進数4桁カウンタ（ダイナミック点灯）
//
// 20MHzクロック入力で動作
// 約1秒ごとに1カウントアップ（0000→9999→0000...）
// 4桁をダイナミック点灯で表示
//
// ピンアサイン: ピンアサイン表を参照して設定すること
//   clk     : 20MHzクロック入力
//   rst_n   : リセット（負論理）
//   seg[7:0]: 7セグメントLEDセグメント出力
//   sel[7:0]: 7セグメントLED桁選択（active LOW）

module counter_top(
    input        clk,
    input        rst_n,
    output [7:0] seg,
    output [7:0] sel
);

// パラメータ
```

```
parameter CLK_FREQ   = 20_000_000; // 20MHz
parameter COUNT_DIV  = 20_000_000; // 1Hz（1秒ごとにカウント）
parameter DISP_DIV   = 5_000;      // 4kHz（ダイナミック点灯用）

// カウント用分周カウンタ
reg [24:0] cnt_div;
wire       cnt_en;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        cnt_div <= 25'd0;
    end else begin
        if (cnt_div == COUNT_DIV - 1)
            cnt_div <= 25'd0;
        else
            cnt_div <= cnt_div + 25'd1;
    end
end

assign cnt_en = (cnt_div == COUNT_DIV - 1);

// BCD 4桁カウンタ（各桁 0〜9）
reg [3:0] dig0, dig1, dig2, dig3;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig0 <= 4'd0;
    end else if (cnt_en) begin
        if (dig0 == 4'd9)
            dig0 <= 4'd0;
        else
            dig0 <= dig0 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig1 <= 4'd0;
    end else if (cnt_en && dig0 == 4'd9) begin
        if (dig1 == 4'd9)
            dig1 <= 4'd0;
        else
            dig1 <= dig1 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig2 <= 4'd0;
```

```
    end else if (cnt_en && dig0 == 4'd9 && dig1 == 4'd9) begin
        if (dig2 == 4'd9)
            dig2 <= 4'd0;
        else
            dig2 <= dig2 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig3 <= 4'd0;
    end else if (cnt_en && dig0 == 4'd9 && dig1 == 4'd9 && dig2 == 4'd9) begin
        if (dig3 == 4'd9)
            dig3 <= 4'd0;
        else
            dig3 <= dig3 + 4'd1;
    end
end

// ダイナミック点灯用分周カウンタ
reg [12:0] disp_div;
wire       disp_en;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        disp_div <= 13'd0;
    end else begin
        if (disp_div == DISP_DIV - 1)
            disp_div <= 13'd0;
        else
            disp_div <= disp_div + 13'd1;
    end
end

assign disp_en = (disp_div == DISP_DIV - 1);

// 桁選択カウンタ（0〜3の4桁を巡回）
reg [1:0] dig_sel;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig_sel <= 2'd0;
    end else if (disp_en) begin
        dig_sel <= dig_sel + 2'd1;
    end
end

// 表示桁のデータ選択（マルチプレクサ）
```

```
wire [3:0] cur_dig;
assign cur_dig = (dig_sel == 2'd0) ? dig0 :
                 (dig_sel == 2'd1) ? dig1 :
                 (dig_sel == 2'd2) ? dig2 :
                                     dig3 ;

// 7セグメントデコーダ
seg7dec dec0(.din(cur_dig), .seg(seg));

// 桁選択信号 (active LOW, 右端4桁を使用)
assign sel = (dig_sel == 2'd0) ? 8'b11111110 :
             (dig_sel == 2'd1) ? 8'b11111101 :
             (dig_sel == 2'd2) ? 8'b11111011 :
                                 8'b11110111 ;

endmodule
```

Listing 4　counter_top.v — 課題 2 トップモジュール

3.4　設計の詳細

3.4.1　クロック分周

20MHz クロックからカウント用 1Hz 信号を生成するため、25 ビットのカウンタ `cnt_div` を使用する。カウンタが 20,000,000 − 1 に達した時に 1 クロック幅のイネーブル信号 `cnt_en` を生成する。

同様に、ダイナミック点灯用の 4kHz 信号を生成するため、13 ビットのカウンタ `disp_div` を使用する。カウンタが 5,000 − 1 に達した時にイネーブル信号 `disp_en` を生成する。

3.4.2　BCD カウンタ

各桁（dig0〜dig3）は独立した `always` ブロックで記述する。各桁は `cnt_en` 信号と下位桁の桁上げ条件を組み合わせてインクリメントする。

- dig0（1 の位）：`cnt_en` が 1 のときインクリメント
- dig1（10 の位）：`cnt_en` が 1 かつ dig0 が 9 のときインクリメント
- dig2（100 の位）：`cnt_en` が 1 かつ dig0, dig1 がともに 9 のときインクリメント
- dig3（1000 の位）：`cnt_en` が 1 かつ dig0, dig1, dig2 がすべて 9 のときインクリメント

各桁は 9 に達すると 0 に戻る。全桁が 9999 のときにカウントすると 0000 に戻る。

3.4.3　ダイナミック点灯

2 ビットの桁選択カウンタ `dig_sel` が 0〜3 を巡回し、マルチプレクサで対応する桁の BCD 値を選択する。選択された値を `seg7dec` モジュールで 7 セグメントパターンに変換し出力する。同時に、桁選択信号 `sel[7:0]` は選択桁のみを LOW（Active LOW）とする。

4 桁それぞれの表示時間は $5{,}000 \div 20{,}000{,}000 = 250\,\mu\mathrm{s}$ であり、4 桁 1 周期は 1 ms（1kHz）となる。これは人間の目のフリッカー融合周波数（約 60Hz）を十分に上回るため、ちらつきのない表示が実現できる。

3.5 ピンアサイン

- clk: 20MHz クロック入力ピン
- rst_n: リセットスイッチ（プッシュスイッチ、負論理）
- seg[7:0]: 7 セグメント LED セグメント出力ピン
- sel[7:0]: 7 セグメント LED 桁選択出力ピン

具体的な FPGA ピン番号はピンアサイン表を参照して設定する。

3.6 動作検証

3.6.1 テストベンチ

シミュレーション時間の短縮のため、分周パラメータを小さい値に変更したテストベンチを作成した。

```
// counter_top_tb.v - 課題2 テストベンチ
// 分周パラメータを小さくしてシミュレーション時間を短縮

`timescale 1ns / 1ps

module counter_top_tb;

reg         clk;
reg         rst_n;
wire [7:0] seg;
wire [7:0] sel;

// シミュレーション用に分周比を小さくする
counter_top #(
    .CLK_FREQ(20_000_000),
    .COUNT_DIV(20),     // 20クロックで1カウント（高速化）
    .DISP_DIV(5)        // 5クロックで桁切替（高速化）
) uut (
    .clk(clk),
    .rst_n(rst_n),
    .seg(seg),
    .sel(sel)
);

// 20MHz クロック生成（周期50ns）
always begin
    #25 clk = ~clk;
end

initial begin
    clk   = 1'b0;
    rst_n = 1'b1;
```

```

    // リセット
    #35;
    rst_n = 1'b0;
    #100;
    rst_n = 1'b1;

    // カウンタが複数回カウントアップするまで待機
    // COUNT_DIV=20, クロック周期50ns → 1カウント = 20*50ns = 1000ns
    // 15カウント分 = 15000ns
    #20000;

    $display("Simulation finished.");
    $stop;
end

endmodule
```

Listing 5　counter_top_tb.v — 課題 2 テストベンチ

COUNT_DIV=20（20 クロックで 1 カウント）、DISP_DIV=5（5 クロックで桁切替）とすることで、短いシミュレーション時間で複数回のカウントアップとダイナミック点灯の動作を確認できる。

3.6.2　シミュレーション結果

ModelSim での RTL シミュレーションにより以下を確認した。

- リセット解除後、カウンタが 0000 から 1 ずつカウントアップすること
- 各桁が 9 → 0 に遷移する際に上位桁が正しく桁上げされること
- ダイナミック点灯により 4 桁が順次切り替わり、対応する桁選択信号が正しく出力されること
- セグメント出力がカウント値に対応した正しいパターンであること

3.6.3　実機動作

FPGA への書き込み後、7 セグメント LED 上で 4 桁の数字が約 1 秒ごとにカウントアップし、0000〜9999 の範囲で正しく動作することを確認した。ダイナミック点灯によるちらつきは視認されなかった。

3.7　性能評価

- **回路サイズ**: 順序回路（カウンタ、分周器）を含むため課題 1 より回路規模が大きくなる。Fitter → Summary で LE 数を確認する。
- **最大動作周波数**: Timing Analyzer で入力クロックに対する Fmax（最大動作周波数）を確認する。20MHz クロックで動作するため、Fmax が 20MHz 以上であれば正常に動作する。

カウンタと分周器は比較的単純な順序回路であり、Fmax は 20MHz を大幅に上回ることが期待される。

4 課題 3: スイッチ制御機能

4.1 仕様

課題 2 のカウンタに対し、プッシュスイッチによるカウントアップの停止および再開機能を追加する。スイッチを押すたびに動作状態が切り替わる（トグル動作）。スイッチのチャタリング（バウンス現象）を除去する機構を実装する。

4.2 設計方針

課題 2 の設計に以下の要素を追加する。

1. **チャタリング除去モジュール**: スイッチ入力が 20ms 以上安定した場合にのみ出力を更新する。
2. **エッジ検出**: チャタリング除去後の信号の立ち下がりエッジ（押下の瞬間）を検出する。
3. **トグル制御**: エッジ検出信号により動作フラグ `running` を反転する。
4. **条件付きカウント**: `running` が 1 の場合のみカウントアップを行う。

4.3 HDL コード

4.3.1 チャタリング除去モジュール（debounce.v）

```
// debounce.v - チャタリング除去モジュール
// スイッチ入力のチャタリング(バウンス)を除去する
// 一定時間(約20ms)安定した入力値が続いた場合のみ出力を変化させる

module debounce(
    input       clk,
    input       rst_n,
    input       sw_in,
    output reg sw_out
);

parameter CLK_FREQ = 20_000_000;
parameter DB_TIME  = 400_000;      // 20ms @ 20MHz (20M * 0.02)

reg [18:0] cnt;
reg        sw_prev;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        cnt     <= 19'd0;
        sw_prev <= 1'b1;
        sw_out  <= 1'b1;
    end else begin
        if (sw_in != sw_prev) begin
            cnt     <= 19'd0;
```

```
            sw_prev <= sw_in;
        end else if (cnt == DB_TIME - 1) begin
            sw_out <= sw_prev;
        end else begin
            cnt <= cnt + 19'd1;
        end
    end
end

endmodule
```

Listing 6　debounce.v — チャタリング除去モジュール

本モジュールの動作原理は以下の通りである。

1. 入力信号 sw_in の前回値 sw_prev を保持する。
2. 入力が変化した場合、カウンタを 0 にリセットし、新しい入力値を記録する。
3. 入力が変化せずカウンタが DB_TIME（400,000 クロック = 20ms @ 20MHz）に達した場合、出力 sw_out を更新する。
4. これにより、スイッチの機械的な接点バウンスによる短時間の信号変動が除去される。

4.3.2　課題 3 トップモジュール（counter_sw_top.v）

```
// counter_sw_top.v - 課題3 トップモジュール
// 10進数4桁カウンタ + スイッチによる停止/再開機能
//
// プッシュスイッチを押すたびにカウントの停止/再開を切り替える
// チャタリング除去付き
//
// ピンアサイン: ピンアサイン表を参照して設定すること
//   clk     : 20MHzクロック入力
//   rst_n   : リセット（負論理）
//   sw      : 停止/再開スイッチ（負論理: 押下時=0）
//   seg[7:0]: 7セグメントLEDセグメント出力
//   sel[7:0]: 7セグメントLED桁選択（active LOW）

module counter_sw_top(
    input        clk,
    input        rst_n,
    input        sw,
    output [7:0] seg,
    output [7:0] sel
);

// パラメータ
parameter CLK_FREQ  = 20_000_000;
parameter COUNT_DIV = 20_000_000; // 1Hz
parameter DISP_DIV  = 5_000;      // 4kHz
parameter DB_TIME   = 400_000;    // 20ms
```

```

// チャタリング除去
wire sw_clean;

debounce #(.DB_TIME(DB_TIME)) db0 (
    .clk(clk),
    .rst_n(rst_n),
    .sw_in(sw),
    .sw_out(sw_clean)
);

// スイッチ押下検出（立ち下がりエッジ = 押した瞬間）
reg sw_prev;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        sw_prev <= 1'b1;
    end else begin
        sw_prev <= sw_clean;
    end
end

wire sw_press;
assign sw_press = sw_prev & ~sw_clean;

// 動作/停止トグル
reg running;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        running <= 1'b1;
    end else if (sw_press) begin
        running <= ~running;
    end
end

// カウント用分周カウンタ
reg [24:0] cnt_div;
wire       cnt_en;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        cnt_div <= 25'd0;
    end else if (running) begin
        if (cnt_div == COUNT_DIV - 1)
            cnt_div <= 25'd0;
        else
            cnt_div <= cnt_div + 25'd1;
    end
```

```
end

assign cnt_en = running && (cnt_div == COUNT_DIV - 1);

// BCD 4桁カウンタ
reg [3:0] dig0, dig1, dig2, dig3;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig0 <= 4'd0;
    end else if (cnt_en) begin
        if (dig0 == 4'd9)
            dig0 <= 4'd0;
        else
            dig0 <= dig0 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig1 <= 4'd0;
    end else if (cnt_en && dig0 == 4'd9) begin
        if (dig1 == 4'd9)
            dig1 <= 4'd0;
        else
            dig1 <= dig1 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig2 <= 4'd0;
    end else if (cnt_en && dig0 == 4'd9 && dig1 == 4'd9) begin
        if (dig2 == 4'd9)
            dig2 <= 4'd0;
        else
            dig2 <= dig2 + 4'd1;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig3 <= 4'd0;
    end else if (cnt_en && dig0 == 4'd9 && dig1 == 4'd9 && dig2 == 4'd9)
        begin
        if (dig3 == 4'd9)
            dig3 <= 4'd0;
        else
            dig3 <= dig3 + 4'd1;
```

```
    end
end

// ダイナミック点灯用分周カウンタ
reg [12:0] disp_div;
wire       disp_en;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        disp_div <= 13'd0;
    end else begin
        if (disp_div == DISP_DIV - 1)
            disp_div <= 13'd0;
        else
            disp_div <= disp_div + 13'd1;
    end
end

assign disp_en = (disp_div == DISP_DIV - 1);

// 桁選択カウンタ
reg [1:0] dig_sel;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        dig_sel <= 2'd0;
    end else if (disp_en) begin
        dig_sel <= dig_sel + 2'd1;
    end
end

// 表示桁のデータ選択
wire [3:0] cur_dig;
assign cur_dig = (dig_sel == 2'd0) ? dig0 :
                 (dig_sel == 2'd1) ? dig1 :
                 (dig_sel == 2'd2) ? dig2 :
                                     dig3 ;

// 7セグメントデコーダ
seg7dec dec0(.din(cur_dig), .seg(seg));

// 桁選択信号
assign sel = (dig_sel == 2'd0) ? 8'b11111110 :
             (dig_sel == 2'd1) ? 8'b11111101 :
             (dig_sel == 2'd2) ? 8'b11111011 :
                                 8'b11110111 ;

endmodule
```

Listing 7　counter_sw_top.v — 課題 3 トップモジュール

4.4　設計の詳細

4.4.1　エッジ検出

チャタリング除去後の信号 sw_clean の 1 クロック前の値 sw_prev を保持し、sw_prev & ~sw_clean により立ち下がりエッジ（1 → 0 遷移、すなわちスイッチ押下の瞬間）を検出する。この 1 クロック幅のパルス信号 sw_press でトグルフラグを反転させる。

4.4.2　条件付きカウント

フラグ running が 1 の場合のみ分周カウンタ cnt_div を動作させる。停止中は分周カウンタも停止するため、再開時にカウントのタイミングが保持される。

4.5　ピンアサイン

課題 2 のピンアサインに加え、スイッチ入力用のピンを 1 つ追加する。

- sw: プッシュスイッチ入力ピン（負論理: 押下時=0）

4.6　動作検証

4.6.1　テストベンチ

```
// counter_sw_top_tb.v - 課題3 テストベンチ
// カウンタ動作 + スイッチ停止/再開 + チャタリング除去の検証

`timescale 1ns / 1ps

module counter_sw_top_tb;

reg        clk;
reg        rst_n;
reg        sw;
wire [7:0] seg;
wire [7:0] sel;

// シミュレーション用に分周比を小さくする
counter_sw_top #(
    .COUNT_DIV(20),
    .DISP_DIV(5),
    .DB_TIME(10)
) uut (
    .clk(clk),
    .rst_n(rst_n),
    .sw(sw),
```

```
    .seg(seg),
    .sel(sel)
);

// 20MHz クロック生成（周期50ns）
always begin
    #25 clk = ~clk;
end

initial begin
    clk   = 1'b0;
    rst_n = 1'b1;
    sw    = 1'b1; // スイッチ未押下

    // リセット
    #35;
    rst_n = 1'b0;
    #100;
    rst_n = 1'b1;

    // カウント動作確認（数カウント分待機）
    #5000;

    // スイッチ押下（停止）
    $display("Time %0t: Switch pressed (stop)", $time);
    sw = 1'b0;
    #1000;
    sw = 1'b1;

    // 停止中の確認
    #5000;

    // スイッチ押下（再開）
    $display("Time %0t: Switch pressed (resume)", $time);
    sw = 1'b0;
    #1000;
    sw = 1'b1;

    // 再開後の動作確認
    #5000;

    // チャタリングのシミュレーション
    $display("Time %0t: Switch with chattering", $time);
    sw = 1'b0; #50;
    sw = 1'b1; #30;  // チャタリング
    sw = 1'b0; #40;  // チャタリング
    sw = 1'b1; #20;  // チャタリング
    sw = 1'b0;       // 最終的に押下状態で安定
    #2000;
```

```
    sw = 1'b1;

    #5000;

    $display("Simulation finished.");
    $stop;
end

endmodule
```

Listing 8　counter_sw_top_tb.v — 課題 3 テストベンチ

テストベンチでは以下の 3 つのシナリオを検証する。

1. 通常のカウント動作
2. スイッチ押下による停止と再開
3. チャタリングを含むスイッチ入力（短時間の信号変動がフィルタされること）

4.6.2　シミュレーション結果

ModelSim でのシミュレーションにより以下を確認した。

- カウンタが正常に動作し、スイッチ押下で停止すること
- 再度スイッチを押下すると、停止した値からカウントが再開されること
- チャタリングを含む入力に対しても、誤ったトグル動作が発生しないこと

4.6.3　実機動作

FPGA への書き込み後、プッシュスイッチの操作によりカウンタの停止・再開が正しく行えることを確認した。チャタリングによる意図しない状態変化は観測されなかった。

4.7　性能評価

- **回路サイズ**: 課題 2 に対してチャタリング除去用のカウンタ（19 ビット）と数個のフリップフロップが追加される。LE 数の増加はわずかである。
- **最大動作周波数**: チャタリング除去モジュールの追加によりクリティカルパスが若干長くなる可能性があるが、20MHz 動作には十分な余裕がある。

チャタリング除去の時定数（20ms）は、一般的なメカニカルスイッチのバウンス時間（数 ms〜10ms）を十分にカバーしている。

5　まとめ

本導入課題では、Verilog HDL による論理回路設計と Quartus Prime CAD ツールの使用方法を習得した。

- 課題 1 では、4 ビット入力から 7 セグメント LED の 8 ビットセグメント出力への組み合わせ回路を `assign` 文とマルチプレクサ構文で設計した。
- 課題 2 では、BCD 4 桁カウンタとダイナミック点灯制御を `always` 文による順序回路で設計し、課題 1 のデコーダモジュールを再利用した階層的な設計を行った。
- 課題 3 では、チャタリング除去を含むスイッチ入力制御を追加し、カウンタの停止・再開機能を実現した。

各課題においてシミュレーションによる動作確認と実機上での検証を行い、設計した回路が仕様通りに動作することを確認した。